Operator Overloading

and the `<algorithm>` library

THE `Clock` CLASS

- A `Clock` stores a time of day: `hours` from 0 to 23 and `minutes` from 0 to 59
- `print` shows each part with two digits, like `09:05`
- The `const` after `getHours()`, `getMinutes()`, and `print()` means they don't change the clock
- We want to make ==, <, and + work on clocks

```
class Clock {
public:
    Clock(int hours, int minutes) {
        this->hours = hours;
        this->minutes = minutes;
    }
    int getHours() const {
        return hours;
    }
    int getMinutes() const {
        return minutes;
    }
    void print() const {
        cout << setfill('0') << setw(2) << hours
             << ":" << setw(2) << minutes
             << setfill(' ');
    }

private:
    int hours;
    int minutes;
};
```

COMPARING TWO CLOCKS

- `lunch` and `alsoLunch` are separate `Clock` objects
- Both of them are set to 12:00

```
int main() {
    Clock lunch(12, 0);
    Clock alsoLunch(12, 0);
    if (lunch == alsoLunch) {
        cout << "same time" << endl;
    }
}
```

Predict: what happens when we compile and run this?

COMPARING TWO CLOCKS

- `lunch` and `alsoLunch` are separate `Clock` objects
- Both of them are set to 12:00

```
int main() {
    Clock lunch(12, 0);
    Clock alsoLunch(12, 0);
    if (lunch == alsoLunch) {
        cout << "same time" << endl;
    }
}
```

```
01_compare.cpp:33:15:
error: no match for 'operator=='
(operand types are 'Clock' and 'Clock')
```

It doesn't compile. C++ doesn't know what `==` means for a `Clock`, so we have to write it.

The whole message is 164 lines long. Look for the first line with your file's name and `error:` in it. `33:15` means line 33, column 15 (line 33 counts the whole file).

WRITING `isSameTime`

- `isSameTime` is a member function
- `other` is the clock we're comparing with. `const Clock&` means it isn't copied and the function can't change it
- `hours` by itself means this clock's hours. `other.hours` means the other clock's hours
- It returns `true` only when both parts match

```
class Clock {
    ...
    bool isSameTime(const Clock& other) const {
        return hours == other.hours &&
               minutes == other.minutes;
    }
    ...
};
```

```
int main() {
    Clock lunch(12, 0);
    Clock alsoLunch(12, 0);
    Clock meeting(9, 30);
    if (lunch.isSameTime(alsoLunch)) {
        cout << "lunch and alsoLunch match" << endl;
    }
    if (lunch.isSameTime(meeting)) {
        cout << "lunch and meeting match" << endl;
    }
}
```

```
lunch and alsoLunch match
```

YOUR TURN: `isSameHeight`

- Open `height.cpp`. It has a `Height` class with `feet` and `inches`, and `print` shows heights like `5'11"`
- Write `isSameHeight` the same way we wrote `isSameTime`
- In `main`, make `finn` (5'4"), `jake` (5'4"), and `bmo` (1'10")
- Test `finn` against `jake` and against `bmo`, the way we tested `lunch`

should print:

```
finn and jake match
```

RENAMING IT TO `operator==`

- The body of the function stays the same
- `operator==` is the function's new name
- Now `lunch == alsoLunch` calls it

before

```
class Clock {
    ...
    bool isSameTime(const Clock& other) const {
        return hours == other.hours &&
               minutes == other.minutes;
    }
    ...
};
```

```
if (lunch.isSameTime(alsoLunch)) {
    cout << "lunch and alsoLunch match" << endl;
}
```

```
lunch and alsoLunch match
```

after

```
class Clock {
    ...
    bool operator==(const Clock& other) const {
        return hours == other.hours &&
               minutes == other.minutes;
    }
    ...
};
```

```
if (lunch == alsoLunch) {
    cout << "lunch and alsoLunch match" << endl;
}
```

```
lunch and alsoLunch match
```

YOUR TURN: `operator==`

- Rename `isSameHeight` to `operator==`, the way we renamed `isSameTime`
- Change your two tests to use `==`
- The output shouldn't change

should print:

```
finn and jake match
```

WHAT `lunch == alsoLunch` MEANS

- `lunch == alsoLunch` runs `operator==` on `lunch`, with `other` referring to `alsoLunch`
- You could write the same call the long way: `lunch.operator==(alsoLunch)`
- Nobody writes it the long way. It just shows which clock is which

```
int main() {
    Clock lunch(12, 0);
    Clock alsoLunch(12, 0);
    if (lunch.operator==(alsoLunch)) {
        cout << "long way works" << endl;
    }
}
```

```
long way works
```



TANGENT!

objects without names

OBJECTS WITHOUT NAMES

- `Clock noon(12, 0);` makes a clock named `noon`
- `Clock(12, 0)` makes a clock with no name
- It's like writing `5 + 3` instead of making `int x = 5;` first and writing `x + 3`
- Both `if` statements on the right compare `lunch` with a clock set to 12:00

```
int main() {
    Clock lunch(12, 0);
    Clock noon(12, 0);
    if (lunch == noon) {
        cout << "lunch matches noon" << endl;
    }
    if (lunch == Clock(12, 0)) {
        cout << "lunch matches 12:00" << endl;
    }
}
```

```
lunch matches noon
lunch matches 12:00
```

USING AN UNNAMED CLOCK

- You can call a member function on an unnamed object, the same as on a named one

```
int main() {
    Clock(9, 5).print();
    cout << endl;
}
```

Predict: what does this print?

USING AN UNNAMED CLOCK

- You can call a member function on an unnamed object, the same as on a named one

```
int main() {
    Clock(9, 5).print();
    cout << endl;
}
```

```
09:05
```

The unnamed clock only exists for that one line. C++ calls an object like this a **temporary**.

FILLING A VECTOR

- To put clocks in a vector, we could name each one first
- Or we can make them right inside the braces, with no names
- Both vectors hold the same two clocks
- We'll also see `return Clock(...)` in `operator+` later: it makes a new, unnamed clock and returns it

with names

```
int main() {
    Clock first(9, 30);
    Clock second(12, 0);
    vector<Clock> meetings = {first, second};
    cout << meetings.size() << endl;
}
```

without names

```
int main() {
    vector<Clock> meetings = {Clock(9, 30),
                              Clock(12, 0)};
    cout << meetings.size() << endl;
}
```

both programs print 2

TANGENT OVER!

COUNTING MATCHES

- `count` is in `<algorithm>`, along with `find` and `sort`
- `meetings.begin()`, `meetings.end()` means "the whole vector"
- `count` goes through the vector and counts the items that are `==` to `Clock(12, 0)`

```
int main() {
    vector<Clock> meetings = {
        Clock(9, 30), Clock(12, 0),
        Clock(15, 15), Clock(12, 0)
    };
    cout << count(meetings.begin(), meetings.end(),
                  Clock(12, 0)) << endl;
}
```

Predict: what does this print?

COUNTING MATCHES

- `count` is in `<algorithm>`, along with `find` and `sort`
- `meetings.begin()`, `meetings.end()` means "the whole vector"
- `count` goes through the vector and counts the items that are `==` to `Clock(12, 0)`

the same count, written as a loop:

```
int matches = 0;
for (const Clock& m : meetings) {
    if (m == Clock(12, 0)) {
        matches++;
    }
}
cout << matches << endl;
```

```
2
```

```
int main() {
    vector<Clock> meetings = {
        Clock(9, 30), Clock(12, 0),
        Clock(15, 15), Clock(12, 0)
    };
    cout << count(meetings.begin(), meetings.end(),
                  Clock(12, 0)) << endl;
}
```

```
2
```

`count` calls `operator==` once for each meeting

without `operator==`, the `count` line doesn't compile

YOUR TURN: `count` ON THE ROSTER

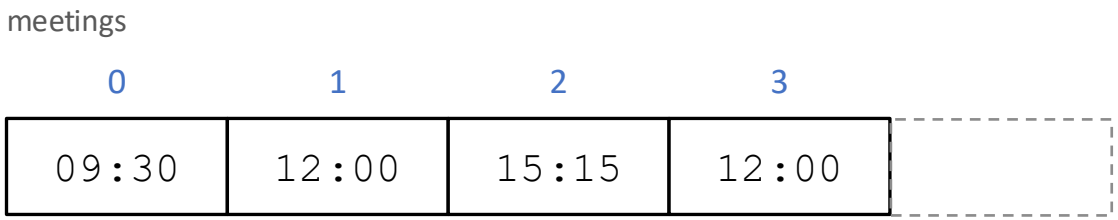
- Make a `vector<Height>` called `roster` holding 6'2", 5'11", 5'4", 6'0", 1'10", and 5'4", in that order
- Use `count` the way we counted meetings at 12:00: print how many people are 5'4", then how many are 7'0"
- `count` uses your `operator==`, so there's nothing new to write in the class

your new code should print:

```
2
0
```

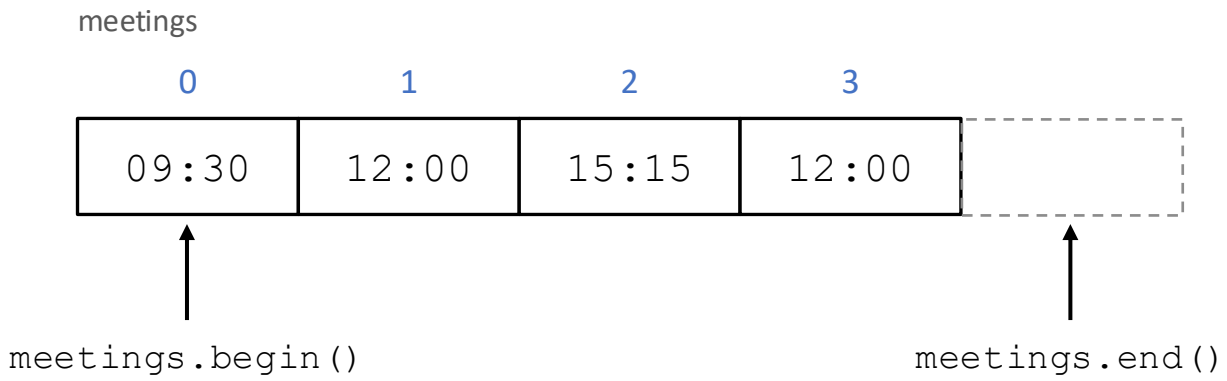
WHERE IS THE 12:00 MEETING?

- `count` tells us how many meetings are at 12:00, but not where they are
- `find` tells us where the first one is
- `find` is in `<algorithm>`, like `count` and `sort`
- `find` uses `==` too, so it works on `Clock` now



`find` GIVES BACK A POSITION

- `find` doesn't give back an index. It gives back a position in the vector. C++ calls a position like this an **iterator**
- `meetings.begin()` is the position of the first meeting
- `meetings.end()` is the position just after the last meeting. Nothing is stored there
- If `find` doesn't find a match, it gives back `meetings.end()`



USING `find`

- `auto` means "C++, you figure out the type"
- We use it here because the actual type is annoyingly complex: `vector<Clock>::iterator`
- `spot != meetings.end()` means "it's in there"
- `spot - meetings.begin()` is how far `spot` is from the start of the vector, which is its index
- Once you have the index, `meetings[index]` works the way it always has

Predict: what does this print?

```
int main() {
    vector<Clock> meetings = {
        Clock(9, 30), Clock(12, 0),
        Clock(15, 15), Clock(12, 0)
    };
    auto spot = find(meetings.begin(),
                     meetings.end(),
                     Clock(12, 0));
    if (spot != meetings.end()) {
        int index = spot - meetings.begin();
        cout << "found at index " << index << endl;
    } else {
        cout << "not found" << endl;
    }
}
```

USING `find`

- `auto` means "C++, you figure out the type"
- We use it here because the actual type is annoyingly complex: `vector<Clock>::iterator`
- `spot != meetings.end()` means "it's in there"
- `spot - meetings.begin()` is how far `spot` is from the start of the vector, which is its index
- Once you have the index, `meetings[index]` works the way it always has

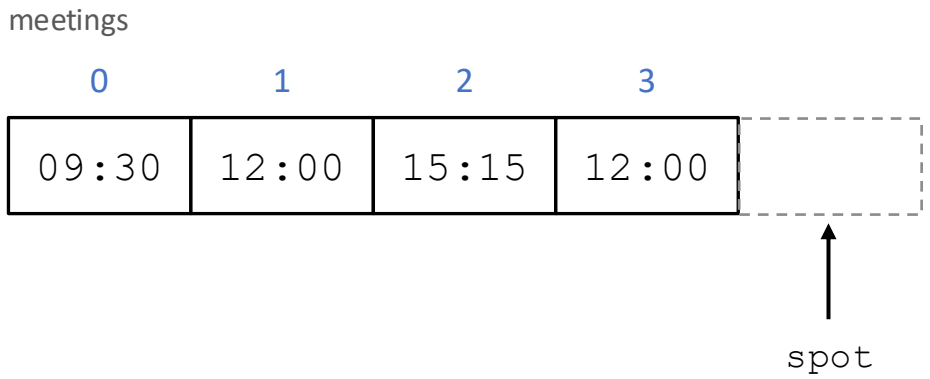
`find` stops at the first match. `count` said there were 2, but `find` gives the position of the first one

```
int main() {
    vector<Clock> meetings = {
        Clock(9, 30), Clock(12, 0),
        Clock(15, 15), Clock(12, 0)
    };
    auto spot = find(meetings.begin(),
                     meetings.end(),
                     Clock(12, 0));
    if (spot != meetings.end()) {
        int index = spot - meetings.begin();
        cout << "found at index " << index << endl;
    } else {
        cout << "not found" << endl;
    }
}
```

```
found at index 1
```

WHEN IT ISN'T THERE

- There's no meeting at 8:00, so `find` gives back `meetings.end()`
- Always check `spot != meetings.end()` before you subtract



without the check, `spot - meetings.begin()` would be 4: the size of the vector, not an index. `meetings[4]` doesn't exist

```
int main() {
    vector<Clock> meetings = {
        Clock(9, 30), Clock(12, 0),
        Clock(15, 15), Clock(12, 0)
    };
    auto spot = find(meetings.begin(),
                     meetings.end(),
                     Clock(8, 0));
    if (spot != meetings.end()) {
        int index = spot - meetings.begin();
        cout << "found at index " << index << endl;
    } else {
        cout << "not found" << endl;
    }
}
```

```
not found
```

YOUR TURN: `find` ON THE ROSTER

- Use `find` on `roster` the way we found the 12:00 meeting
- Look for someone 5'11" tall. Print `found at index` and the index, or `not found`
- Then do the same for 7'0". Nobody is that tall

your new code should print:

```
found at index 1
not found
```

WHICH CLOCK COMES FIRST?

- `sort` puts things in order using `<`, so `Clock` needs an `operator<`
- Here's a first try: one clock is earlier than another when its hours and its minutes are both smaller

Predict: what does this print?

```
class Clock {
    ...
    bool operator<(const Clock& other) const {
        return hours < other.hours &&
               minutes < other.minutes;
    }
    ...
};
```

```
int main() {
    Clock early(9, 45);
    Clock late(10, 15);
    if (early < late) {
        cout << "9:45 comes first" << endl;
    } else {
        cout << "9:45 does not come first" << endl;
    }
}
```

WHICH CLOCK COMES FIRST?

- `sort` puts things in order using `<`, so `Clock` needs an `operator<`
- Here's a first try: one clock is earlier than another when its hours and its minutes are both smaller

```
class Clock {
    ...
    bool operator<(const Clock& other) const {
        return hours < other.hours &&
               minutes < other.minutes;
    }
    ...
};
```

```
int main() {
    Clock early(9, 45);
    Clock late(10, 15);
    if (early < late) {
        cout << "9:45 comes first" << endl;
    } else {
        cout << "9:45 does not come first" << endl;
    }
}
```

```
9:45 does not come first
```

`9 < 10` is true, but `45 < 15` is false, so the `&&` makes the whole thing false

9:45 really is earlier than 10:15, so this `operator<` is wrong

COMPARING HOURS, THEN MINUTES

- If the hours are different, compare hours
- We look at minutes only if the hours are the same

```
class Clock {
    ...
    bool operator<(const Clock& other) const {
        if (hours != other.hours) {
            return hours < other.hours;
        }
        return minutes < other.minutes;
    }
    ...
};
```

with the same `main` as the last slide:

```
9:45 comes first
```

YOUR TURN: < FOR `Height`

- Add `operator<` to `Height` the way we did for `Clock`: compare feet first, then inches
- Test whether 5'11" < 6'2", and whether 6'2" < 5'11". When one is true, print it like `5'11" is shorter than 6'2"`

your new code should print:

```
5'11" is shorter than 6'2"
```

SORTING MEETINGS

- `sort` is in `<algorithm>` too
- `sort(meetings.begin(), meetings.end())` sorts the whole vector
- `sort` decides the order by comparing pairs of meetings with `operator<`

```
int main() {
    vector<Clock> meetings = {
        Clock(14, 0), Clock(9, 30),
        Clock(12, 0), Clock(9, 5)
    };
    sort(meetings.begin(), meetings.end());
    for (Clock m : meetings) {
        m.print();
        cout << endl;
    }
}
```

Predict: what does this print?

SORTING MEETINGS

- `sort` is in `<algorithm>` too
- `sort(meetings.begin(), meetings.end())` sorts the whole vector
- `sort` decides the order by comparing pairs of meetings with `operator<`

```
int main() {
    vector<Clock> meetings = {
        Clock(14, 0), Clock(9, 30),
        Clock(12, 0), Clock(9, 5)
    };
    sort(meetings.begin(), meetings.end());
    for (Clock m : meetings) {
        m.print();
        cout << endl;
    }
}
```

```
09:05
09:30
12:00
14:00
```

with the first try at `operator<`, this prints 14:00, 09:30, 12:00, 09:05, in the order they started in

YOUR TURN: `sort` THE ROSTER

- Sort `roster` the way we sorted meetings, and print each height on its own line
- `sort` uses your `operator<`, so there's nothing new to write in the class

your new code should print:

```
1'10"
5'4"
5'4"
5'11"
6'0"
6'2"
```

ADDING MINUTES TO A CLOCK

- `lunch + 90` should mean "90 minutes after lunch"
- The parameter is an `int` indicating how many minutes to add
- The function turns the time into minutes, adds, and turns the total back into hours and minutes
- `%` gives the remainder, so `% (24 * 60)` keeps the total under one day

```
class Clock {
    ...
    Clock operator+(int extra) const {
        int total = hours * 60 + minutes + extra;
        total = total % (24 * 60);
        return Clock(total / 60, total % 60);
    }
    ...
};
```

```
int main() {
    Clock lunch(12, 0);
    Clock late(23, 50);
    Clock a = lunch + 90;
    Clock b = late + 15;
    a.print();
    cout << endl;
    b.print();
    cout << endl;
}
```

YOUR TURN: + FOR `Height`

- Add `operator+` so that `Height + Height` gives the total height
- Inches roll over at 12, the way minutes roll over at 60. A height doesn't wrap around the way a clock does
- Let's say Finn (5'8") stands on a box that's 7 inches tall. Print that total, then 2'6" + 3'6"

your new code should print:

```
6'3"
6'0"
```

WHY THERE'S NO `Clock + Clock`

- We wrote `Clock + int`, not `Clock + Clock`
- What would 12:00 + 18:00 mean? The answer isn't a time of day
- We only write an operator when it's clear what it should do

```
int main() {
    Clock lunch(12, 0);
    Clock dinner(18, 0);
    Clock both = lunch + dinner;
}
```

```
error: no match for 'operator+' (operand
types are 'Clock' and 'Clock')
```

Discuss: what should `Height(5, 8) + 3` mean?

STORING TOTAL MINUTES

- What if `Clock` stored one number, the minutes since midnight?
- Each operator gets shorter, and `operator+` no longer has to convert
- `getHours`, `getMinutes`, and `print` work out hours and minutes when they need them
- Code that uses `Clock` doesn't change. The sorting and adding tests print the same thing
- This is the question from the `Date` reflection: what should the class store?

```
class Clock {
public:
    Clock(int hours, int minutes) {
        totalMinutes = hours * 60 + minutes;
    }
    int getHours() const {
        return totalMinutes / 60;
    }
    int getMinutes() const {
        return totalMinutes % 60;
    }
    void print() const {
        cout << setfill('0') << setw(2) << getHours()
             << ":" << setw(2) << getMinutes()
             << setfill(' ');
    }
    ...
};
```

```
class Clock {
    ...
    bool operator==(const Clock& other) const {
        return totalMinutes == other.totalMinutes;
    }
    bool operator<(const Clock& other) const {
        return totalMinutes < other.totalMinutes;
    }
    Clock operator+(int extra) const {
        return Clock(0, (totalMinutes + extra) % (24 * 60));
    }

private:
    int totalMinutes;
};
```

YOUR TURN: STORING TOTAL INCHES

- Save a copy of your working `height.cpp` first
- Change `Height` to store one `int totalInches` instead of `feet` and `inches`, the way `Clock` now stores `totalMinutes`
- Update the constructor, the getters, `print`, and the operators
- Don't change `main`. It should print exactly what it printed before

the whole program should still print:

```
finn and jake match
finn is 5'4"
2
0
found at index 1
not found
5'11" is shorter than 6'2"
1'10"
5'4"
5'4"
5'11"
6'0"
6'2"
6'3"
6'0"
```

WHAT WE TAUGHT C++

- Each operator is a member function with a special name
- `count` uses `==`, and `sort` uses `<`

When you write	C++ calls	Used by
`lunch == alsoLunch`	`lunch.operator==(alsoLunch)`	`count`
`early < late`	`early.operator<(late)`	`sort`
`lunch + 90`	`lunch.operator+(90)`	our own code

QUESTIONS YOU MIGHT BE ASKING

How can `operator==` read `other.hours`? It's private.

`private` means private to the class, not to one object. Any `Clock` function can use any `Clock`'s private variables.

Does `!=` work now?

If using C++17: you write it too, using the `==` you already have:
```
return !(*this == other);
```
If using C++20: C++ does this for you

Can I write `90 + lunch`?

Not yet. The left side of `+` is an `int`, and our `operator+` belongs to `Clock`. That takes a function outside the class, which comes later.

Why doesn't `lunch + 90` change `lunch`?

`+` gives back a new `Clock`, the same way `x + 1` doesn't change `x`. Changing `lunch` itself would be `+=`, which we haven't written.

Does `sort` need `==`?

No. `sort` only uses `<`.

Do I need the `const` at the end?

Keep it. It says the function doesn't change the clock, and without it, `==` doesn't compile when the left side is a `const Clock`.